

Article Title

First Author^{1,2*}, Second Author^{2,3†} and Third Author^{1,2†}

^{1*}Department, Organization, Street, City, 100190, State, Country.

²Department, Organization, Street, City, 10587, State, Country.

³Department, Organization, Street, City, 610101, State, Country.

*Corresponding author(s). E-mail(s): iauthor@gmail.com;
Contributing authors: iiauthor@gmail.com; iiiauthor@gmail.com;

[†]These authors contributed equally to this work.

Abstract

Recommender systems deployed in dynamic environments must adapt to users whose preferences drift over time, yet many bandit algorithms still assume stationary reward distributions. This work presents a reproducible simulation framework that models user drift as a latent preference vector over content categories evolving in a non-stationary multi-armed bandit setting. Within this framework we implement four adaptive algorithms—Non-Stationary ϵ -Greedy, Sliding-Window UCB, CUSUM-UCB, and EXP3.S—and evaluate them on synthetically generated drifting-user data using average and cumulative reward with confidence intervals over 50 runs. The results show that Sliding-Window UCB consistently achieves the highest long-run reward, followed by CUSUM-UCB and Non-Stationary ϵ -Greedy, while the adversarially robust EXP3.S underperforms in this stochastic drift regime. These findings suggest that methods which explicitly discount or reset outdated observations are well suited to gradual user drift, and that adversarial bandit algorithms can be overly conservative when non-stationarity arises from user behavior rather than malicious manipulation.

Keywords: Non-stationary multi-armed bandits, User preference drift, Recommender systems, Sliding-Window UCB, CUSUM-UCB, Non-stationary epsilon-greedy, EXP3.S adversarial bandit, Exploration–exploitation trade-off

1 Introduction

Modern recommender systems must adapt to users whose preferences change over time, a phenomenon commonly referred to as user drift. User preferences change independently of the system’s actions. When the environment evolves on its own rather than through action-induced state transitions, the problem aligns more closely with a non-stationary multi-armed bandit setting rather than a multi-state Markov decision process.

To investigate this class of problems in a controlled and reproducible manner, this research constructs synthetic user environments in which each user is represented by a latent preference vector over a set of content categories or arms. In the initial timestep, users begin without inherent bias between arms, matching the initialization used in the drift environment defined in `env.py`. These latent vectors determine the initial affinities of each user and serve as the basis for reward generation.

User drift is simulated by updating each user’s latent affinity vector at every timestep. The vector is normalized to preserve a valid normal probability distribution. This process captures gradual or abrupt changes in user interest that occur naturally in real-world applications due to evolving tastes, external trends, or shifting contexts. Importantly, this drift occurs independently of the agent’s actions, which reinforces the interpretation of the recommendation task as a single-state non-stationary bandit problem.

Because user preferences evolve over time, algorithms must adapt quickly to changes in reward distributions. This study evaluates four adaptive bandit algorithms: Non-Stationary Epsilon-Greedy, Sliding Window UCB (SW-UCB), CUSUM-UCB, and EXP3.S. Each algorithm processes the same synthetically generated datasets that include drift in user preferences. This controlled setup ensures reproducibility and creates a fair basis for performance comparison. The objective is to examine how effectively each algorithm can detect and track the shifting reward landscape and to determine which methods are most suitable for recommendation tasks in dynamic environments.

2 Literature Review

A wide range of strategies has been developed to address non-stationarity in multi-armed bandit problems, particularly in settings where the reward distribution drifts or changes abruptly over time. Classical bandit algorithms such as UCB and ϵ -greedy assume stationary reward distributions and therefore struggle when user preferences evolve, as their estimates place equal weight on outdated and recent observations. To mitigate this limitation, several adaptive variants have been proposed in the literature, each incorporating mechanisms to discount stale information, detect distributional shifts, or explicitly reset statistics after suspected changes.

This section reviews four representative approaches that form the basis of our comparative study: Sliding-Window UCB (SW-UCB), Non-Stationary ϵ -Greedy, EXP3.S, and CUSUM-UCB. These methods span both stochastic and adversarial formulations of non-stationary bandits and provide a spectrum of adaptation mechanisms, including

sliding-window averaging, exponential recency-weighted updates, weight-sharing regularization, and online change detection. By examining these algorithms, we establish a comprehensive foundation for evaluating how different adaptation strategies respond to user drift in recommendation-style environments.

- **Sliding-Window UCB (SW-UCB)**[1]: SW-UCB is a variant of UCB designed for non-stationary environments with abrupt changes. Instead of using all past observations, it computes estimates using only the most recent τ rounds, allowing rapid adaptation to changing reward distributions.

The sliding-window empirical mean for arm i at time t is

$$\bar{X}_t(\tau, i) = \frac{1}{N_t(\tau, i)} \sum_{s=t-\tau+1}^t X_s(i) \mathbb{1}_{\{I_s=i\}},$$

where

$$N_t(\gamma, i) = \sum_{s=1}^t \gamma^{t-s} \mathbb{1}_{\{I_s=i\}},$$

The is the number of times the arm i was selected in the last τ rounds, $\gamma \in (0, 1)$ is the discount factor controlling how quickly past observations are forgotten and ξ is a some appropriate constant.

SW-UCB constructs an upper-confidence bound using:

$$c_t(\tau, i) = B \sqrt{\frac{\xi \log(t \wedge \tau)}{N_t(\tau, i)}},$$

where B is the upper-bound on the rewards, $t \wedge \tau$ denotes the minimum of t and τ and $\xi > 1/2$ controls exploration.

Action selection is performed by

for $t = 1$ to K , play arm $I_t = t$;

for $t = K + 1$ to T , play arm

$$I_t = \arg \max_{1 \leq i \leq K} (\bar{X}_t(\tau, i) + c_t(\tau, i)).$$

After receiving the reward $X_t(a_t)$, the sliding-window statistics are updated by discarding observations older than τ rounds and inserting the new one:

$$N_{t+1}(\gamma, i) = \sum_{s=t-\tau+2}^{t+1} \gamma^{t+1-s} \mathbb{1}_{\{I_s=i\}},$$

$$\bar{X}_{t+1}(\tau, a) = \frac{1}{N_{t+1}(\tau, a)} \sum_{s=t-\tau+2}^{t+1} X_s(i) \mathbb{1}_{\{I_s=i\}},$$

$$c_{t+1}(\tau, i) = B \sqrt{\frac{\xi \log((t+1) \wedge \tau)}{N_{t+1}(\tau, i)}}.$$

- **Non-Stationary Epsilon-Greedy**[2]: This is an adaptation of the classical ε -greedy method for environments in which the reward distributions drift over time. Instead of sample averages, the action-value estimates use a *constant step-size* update, which implements an exponential recency-weighted average that discounts older rewards.

The estimate of the value of arm i at time t is updated according to the incremental constant-step-size rule

$$Q_{t+1}(i) = Q_t(i) + \alpha(R_t - Q_t(i)) \mathbb{1}_{\{I_t=i\}},$$

where $\alpha \in (0, 1]$ is the constant step-size parameter. Unrolling this recursion gives the exponential recency-weighted form

$$Q_{t+1}(i) = (1 - \alpha)^t Q_1(i) + \sum_{s=1}^t \alpha(1 - \alpha)^{t-s} R_s \mathbb{1}_{\{I_s=i\}},$$

showing that recent rewards receive more weight than older ones, with decay factor $(1 - \alpha)$.

Action selection is performed using the ε -greedy policy:

for $t = 1$ to K , play arm $I_t = t$;

for $t = K + 1$ to T , select arm

$$I_t = \begin{cases} \text{a random arm in } \{1, \dots, K\}, & \text{with probability } \varepsilon, \\ \arg \max_{1 \leq i \leq K} Q_t(i), & \text{with probability } 1 - \varepsilon. \end{cases}$$

After observing reward R_t , the value estimates are updated using the constant step-size rule:

$$Q_{t+1}(I_t) = Q_t(I_t) + \alpha(R_t - Q_t(I_t)).$$

Because the estimate $Q_t(i)$ places exponentially decaying weight on past rewards with weights proportional to $(1 - \alpha)^k$ for k -step-old observations method adapts effectively to non-stationary reward distributions. Larger values of α allow faster adaptation, while smaller values provide greater stability.

- **EXP3.S**[3]: EXP3.S is a variant of the EXP3 algorithm proposed in Auer et al. (1995) [4] designed to control regret against *arbitrary* (possibly switching) action sequences. Rather than competing only with the best single arm, EXP3.S adapts to piecewise-stationary environments by incorporating an additional stabilization term in the weight update that helps the algorithm track changing optimal arms. Like EXP3, it maintains exponential weights over the K arms and mixes between exploitation and uniform exploration. However, EXP3.S adds a regularization term proportional to α that penalizes rapid fluctuations and allows regret bounds scaling with the *hardness* of the comparator sequence (i.e., the number of switches throughout the sequence).

Action selection:

At each round t , compute the probability distribution

$$p_i(t) = (1 - \gamma) \frac{w_i(t)}{\sum_{j=1}^K w_j(t)} + \frac{\gamma}{K}, \quad i = 1, \dots, K.$$

Then draw action I_t according to the distribution $p(t)$.

Reward estimation:

After receiving reward $x_{I_t}(t) \in [0, 1]$, define the unbiased importance-weighted estimator

$$\hat{x}_j(t) = \begin{cases} \frac{x_j(t)}{p_j(t)}, & j = I_t, \\ 0, & j \neq I_t. \end{cases}$$

Weight update:

For each arm $j = 1, \dots, K$, update

$$w_j(t+1) = w_j(t) \exp\left(\frac{\gamma}{K} \hat{x}_j(t)\right) + \frac{e\alpha}{K} \sum_{i=1}^K w_i(t).$$

The additive term

$$\frac{e\alpha}{K} \sum_{i=1}^K w_i(t)$$

helps the algorithm adapt to environments where the best arm switches, enabling regret bounds that depend on the number of switches rather than on T alone.

- **CUSUM-UCB**[5]: CUSUM-UCB is a change-detection based bandit algorithm designed for piecewise-stationary environments. It augments the classical UCB index with a two-sided online CUSUM detector that monitors each arm for statistically significant shifts in its reward distribution. When a change is detected on arm i , the algorithm *restarts* that arm by resetting its empirical mean, sample count, and the associated CUSUM statistics. This prevents outdated pre-change rewards from contaminating the UCB estimate and enables rapid recovery after abrupt changes. For each arm, the first M observations are used to estimate the pre-change mean,

$$\hat{u}_0 = \frac{1}{M} \sum_{k=1}^M y_k,$$

and for each subsequent sample y_k , the CUSUM updates use the steps

$$(s_k^+, s_k^-) = (y_k - \hat{u}_0 - \varepsilon, \hat{u}_0 - y_k - \varepsilon) \mathbb{1}_{\{k > M\}},$$

$$g_k^+ = \max\{0, g_{k-1}^+ + s_k^+\}, \quad g_k^- = \max\{0, g_{k-1}^- + s_k^-\}.$$

A change is detected whenever either statistic crosses the threshold h , i.e.,

$$g_k^+ \geq h \quad \text{or} \quad g_k^- \geq h,$$

upon which the corresponding arm is restarted: its restart time τ_i is set to the current round, and the CUSUM variables (g_k^+, g_k^-) are reset to zero.

The UCB part of the algorithm uses only post-restart statistics. Let τ_i denote the last restart time of arm i . Then its valid sample count, empirical mean, and confidence padding at time t are

$$N_t(i) = \sum_{s=\tau_i}^t \mathbb{1}_{\{I_s=i\}}, \quad \bar{X}_t(i) = \frac{1}{N_t(i)} \sum_{s=\tau_i}^t X_s(i) \mathbb{1}_{\{I_s=i\}},$$

$$C_t(i) = \sqrt{\frac{\xi \log n_t}{N_t(i)}}, \quad n_t = \sum_{j=1}^K N_t(j),$$

where $\xi > 0$ is the UCB exploration constant.

Action selection combines UCB exploitation with uniform sampling at rate α , as required to feed the CUSUM detectors:

$$I_t = \begin{cases} \arg \max_{1 \leq i \leq K} (\bar{X}_t(i) + C_t(i)), & \text{with probability } 1 - \alpha, \\ \text{a uniformly random arm in } \{1, \dots, K\}, & \text{with probability } \alpha/K. \end{cases}$$

Update rule: After observing reward $X_t(I_t)$, the algorithm

1. updates the CUSUM statistics $g_k^+, g_k^-, s_k^+, s_k^-$ for the played arm;
2. if a CUSUM alarm is raised, resets $\tau_{I_t} = t + 1$ and clears the CUSUM state;
3. otherwise, updates $(N_t(i), \bar{X}_t(i), C_t(i))$ for the played arm using the post-restart definitions above.

Because all pre-change samples of an arm are discarded upon detection, CUSUM-UCB avoids the slow recovery faced by classical UCB when reward means abruptly decrease.

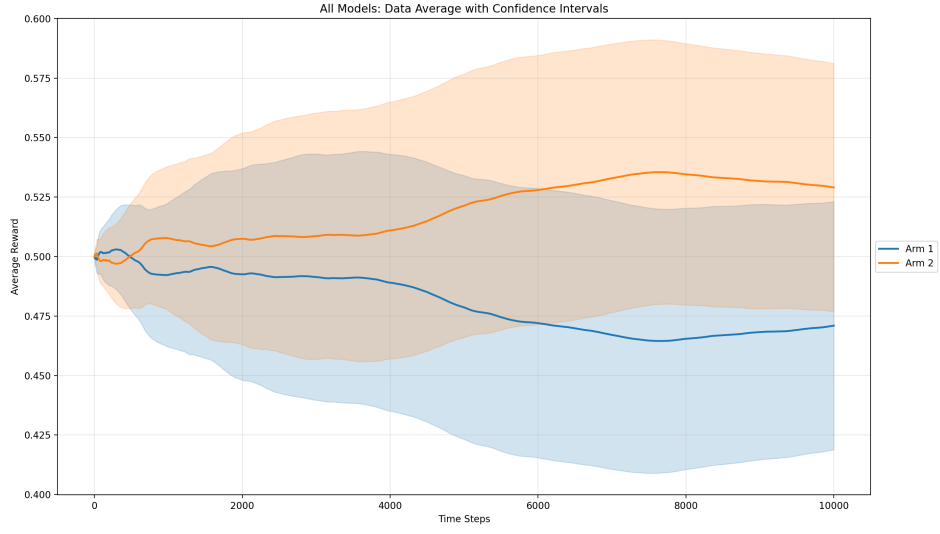
3 Methodology

3.1 Bandit Framework

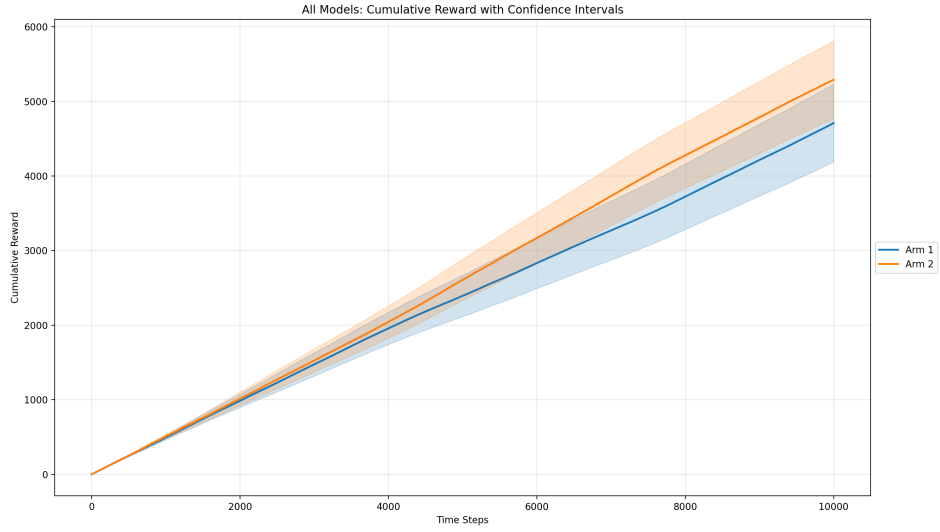
All experiments are conducted in the **drifting** user environment generated by the `create_environment` function, where a single user's preference vector over K documents evolves over time through Gaussian perturbations and renormalization. The number of arms is fixed to $K = 2$, corresponding to two competing documents, and each episode consists of $T = 10,000$ interaction steps. For every algorithm, performance is estimated over 50 independent experiments, each initialized with a distinct random seed drawn from $\{0, \dots, 99\}$. [1–3, 5]

For each model and seed, the runner first samples a new drifting user trajectory, then trains the selected bandit policy with its prescribed hyperparameters (Non-Stationary ε -Greedy with $\varepsilon = 0.05$ and $\alpha = 0.001$; SW-UCB with window size 50; CUSUM-UCB with $\varepsilon = 0.2$, $h = 10$, $\alpha = 0.2$; EXP3.S with $\gamma = 0.07$, $\alpha = 0.1$). At each round the environment emits a reward vector, the agent selects one arm, and only the realized reward on that arm is observed and logged together with a one-hot encoding of the chosen action.

After every run, three NumPy arrays are saved per model and seed: the underlying reward data, the sequence of observed rewards, and a matrix of arm selections. These tensors are later aggregated across all 50 trials to compute mean rolling averages and cumulative rewards, as well as corresponding pointwise 95% confidence intervals that are visualized in Figures [1a](#) and [1b](#)



(a) Rolling average reward of each arm over $T = 10,000$ rounds in the drifting user environment. The solid lines show the mean per-time-step reward across 50 runs, with shaded 95% confidence intervals.



(b) Cumulative reward of each arm over $T = 10,000$ rounds. The mean cumulative reward across 50 runs is shown with shaded 95% confidence intervals, highlighting that Arm 2 becomes dominant as preferences drift.

Fig. 1: Drifting user environment: average and cumulative rewards for both arms with confidence intervals.

3.2 Metrics

The primary objective metric is *cumulative reward*, defined for a policy π over horizon T as

$$R_{\text{cum}}^{\pi}(T) = \sum_{t=1}^T R_t,$$

where R_t is the realized reward at round t . This quantity captures the total utility accrued by the algorithm and is the standard notion of performance in bandit settings. For each model, we report the mean and pointwise 95% confidence intervals of $R_{\text{cum}}^{\pi}(T)$ across the 50 independent runs generated in our experimental setup.[1–3, 5]

To analyze adaptation speed and stability, we also consider the *rolling average reward*

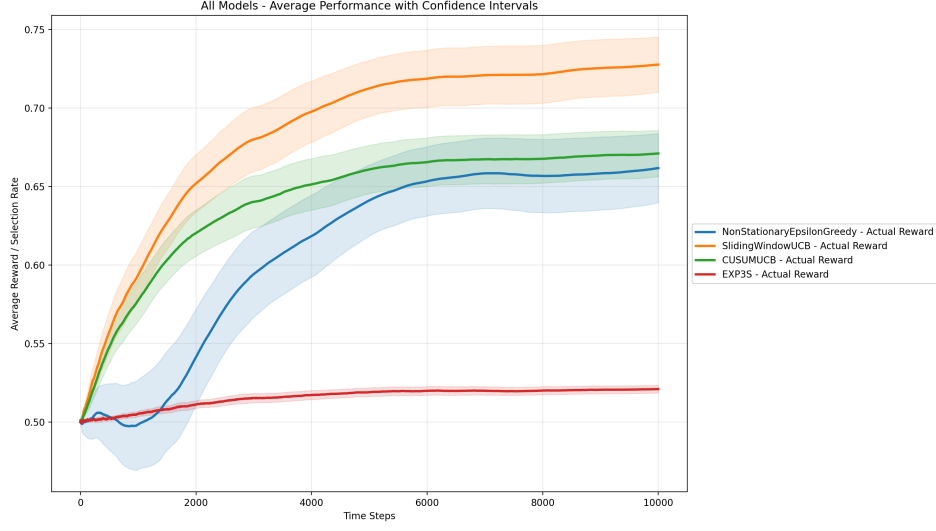
$$\bar{R}^{\pi}(t) = \frac{1}{t} \sum_{s=1}^t R_s, \quad t = 1, \dots, T,$$

which summarizes the per-time-step performance up to round t . The rolling average highlights transient behavior during regime changes, while the cumulative curve emphasizes long-run efficiency. In both cases, we aggregate results over random seeds and estimate confidence bands using the empirical standard error, similar to visualization in Figures 1a and 1b.

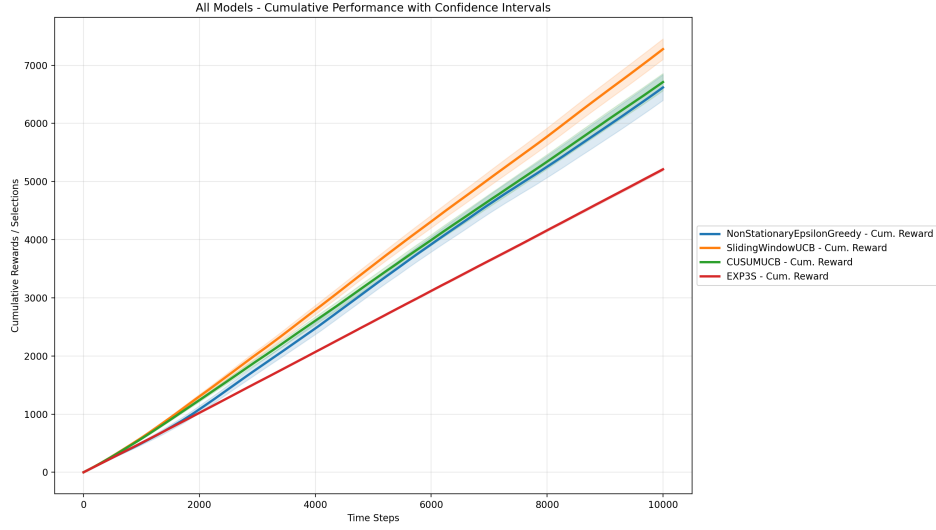
4 Results

Figure 2(a) shows that all stochastic bandit algorithms quickly improve their average reward as they adapt to the drifting user preferences, but they do so at different rates. SlidingWindowUCB exhibits the fastest increase and ultimately attains the highest steady-state performance, followed by CUSUM-UCB and the Non-Stationary ε -Greedy baseline. EXP3.S, which is tailored to adversarial settings, improves more slowly and stabilizes at a noticeably lower average reward.

The cumulative reward curves in Figure 2(b) reflect the same ordering. SlidingWindowUCB accumulates the largest total reward over the 10,000 rounds, confirming its advantage in this smoothly drifting environment, while CUSUM-UCB and Non-Stationary ε -Greedy achieve similar but slightly smaller gains. EXP3.S lags behind throughout the horizon, indicating that its robustness to adversarial sequences comes at the cost of lower efficiency when the rewards are stochastic but non-stationary.



(a) Mean rolling average reward for all four algorithms over $T = 10,000$ rounds, averaged across 50 runs with pointwise 95% confidence intervals.



(b) Mean cumulative reward trajectories for all four algorithms over $T = 10,000$ rounds, averaged across 50 runs with pointwise 95% confidence intervals.

Fig. 2: Comparison of average and cumulative rewards for the evaluated bandit algorithms in the drifting user environment.

5 Discussion

The empirical results highlight clear differences in how the four algorithms cope with gradual drift in user preferences. In the drifting environment, the rolling average rewards show that SlidingWindowUCB adapts most quickly and maintains the highest steady-state performance, which is consistent with its design choice of discarding stale observations via a finite window. CUSUM-UCB also benefits from its explicit change-detection mechanism, recovering rapidly after shifts but exhibiting slightly more variability due to exploratory resets. The Non-Stationary ϵ -Greedy baseline eventually tracks the dominant arm, yet its constant step-size update reacts more slowly than the targeted mechanisms in SW-UCB and CUSUM-UCB.

EXP3.S behaves differently because it is rooted in the adversarial bandit formulation, where rewards are viewed as an arbitrary sequence rather than independent draws from fixed probability distributions. In this paradigm the algorithm maintains exponential weights over arms and samples actions from a mixture of the normalized weights and a uniform exploration distribution, similar in spirit to an entropy-regularized or softmax policy. Only the arm played at each round receives a non-zero importance-weighted reward estimate, so the corresponding weight is updated while the others remain unchanged. This update structure implicitly favors previously selected actions and is well suited to worst-case regret guarantees, but in the smoothly drifting stochastic environment considered here it leads to conservative adaptation and lower average reward than the stochastic baselines.

6 Conclusion

This work introduced a reproducible simulation framework for studying user drift through a non-stationary multi-armed bandit lens and used it to compare four adaptive bandit algorithms. By representing users as drifting preference vectors over content categories, the framework isolates the effect of evolving interests while keeping the interaction model intentionally simple. Within this environment, SlidingWindowUCB consistently achieved the highest average and cumulative rewards, with CUSUM-UCB and Non-Stationary ϵ -Greedy forming a competitive middle tier and EXP3.S lagging behind.

These findings indicate that lightweight stochastic methods that explicitly discount or reset outdated information are well suited to recommendation scenarios where preferences drift gradually over time. In contrast, adversarially oriented algorithms such as EXP3.S may sacrifice efficiency when the true source of non-stationarity is user behavior rather than hostile perturbations. The proposed framework and code base can serve as a testbed for future adaptive bandit designs and for evaluating robustness–efficiency trade-offs under controlled forms of drift.

6.1 Limitations

The experimental setup focuses on a single-user, two-arm setting with a specific Gaussian drift process, which simplifies interpretation but does not capture the full

complexity of real recommender systems. The user’s latent preferences evolve according to a synthetic and symmetric noise process without contextual features, feedback loops, or heterogeneous user populations. Moreover, hyperparameters for each algorithm were chosen using reasonable defaults rather than systematic tuning, so the reported ranking may change under different optimization criteria, time horizons, or drift regimes.

Another limitation is that the evaluation uses only reward-based metrics (average and cumulative reward) and does not consider complementary notions such as dynamic regret with respect to a time-varying oracle, fairness across arms, or robustness to model misspecification. The conclusions are therefore conditional on the specific reward structure and drift dynamics encoded in the simulator, and care must be taken when extrapolating to production systems with richer feedback and business constraints.

6.2 Future Scope

Several directions can extend this work. First, the drifting environment can be generalized to multi-user and multi-category settings with contextual information, enabling the study of contextual and linear bandit algorithms under user drift. Second, future experiments could incorporate abrupt regime changes, seasonality, or mixed stochastic–adversarial reward sequences to test whether the relative advantages of SW-UCB, CUSUM-UCB, and EXP3.S persist under more challenging dynamics. Third, a systematic hyperparameter search and sensitivity analysis would clarify how robust the observed performance ordering is to design choices such as window length, CUSUM thresholds, and exploration rates.

Beyond simulation, the framework could be adapted for offline evaluation using historical interaction logs, for example via inverse propensity scoring or counterfactual estimators, to bridge the gap between synthetic and real-world data. Integrating additional objectives—such as diversity, fairness, or risk control—would also allow the comparison of adaptive bandit algorithms along multiple axes that matter in deployed recommender systems.

References

- [1] Garivier, A., Moulines, E.: On upper-confidence bound policies for non-stationary bandit problems (2008) [arXiv:0805.3415](#) [math.ST]
- [2] Sutton, R.S., Barto, A.G., et al.: Reinforcement learning: An introduction **1**(1) (1998)
- [3] Chen, N., Yang, S., Zhang, H.: Bridging adversarial and nonstationary multi-armed bandit (2023) [arXiv:2201.01628](#) [cs.LG]
- [4] Auer, P., Cesa-Bianchi, N., Freund, Y., Schapire, R.E.: The nonstochastic multi-armed bandit problem. *SIAM J. Comput.* **32**, 48–77 (2002)

- [5] Liu, F., Lee, J., Shroff, N.: A change-detection based framework for piecewise-stationary multi-armed bandit problem (2017) [arXiv:1711.03539](#) [cs.LG]